Machine Learning

Variational Autoencoder

Dong-Kyu Chae

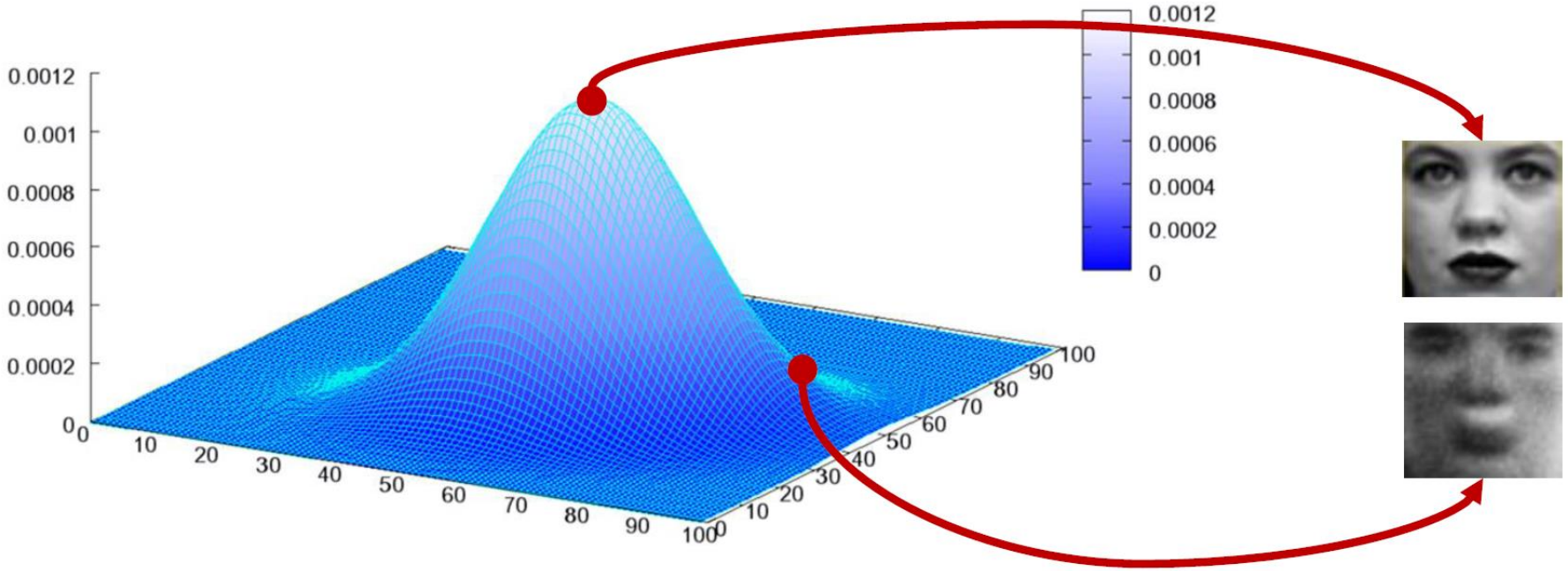
PI of the Data Intelligence Lab. @HYU

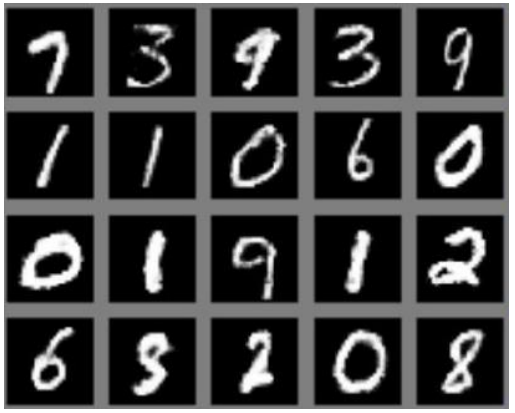
Dept. of Computer and Software

Hanyang University

Machine Learning with Latent Variables

- Generative model learns data distribution of a given dataset





Machine Learning with Latent Variables

Idea

- There are **hidden variables** $\boldsymbol{z}$ behind the **observed data**

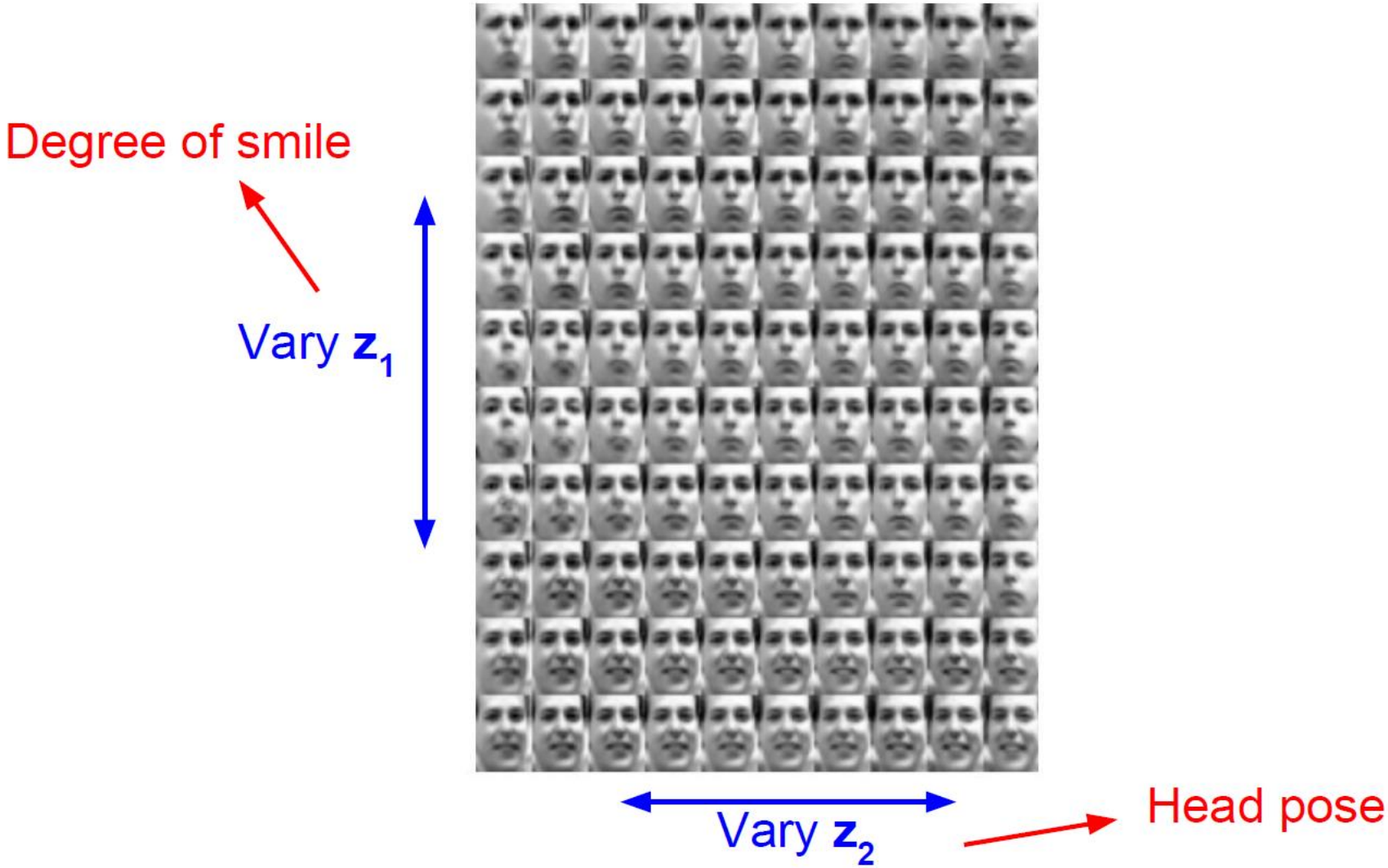
Hidden variables

Observed data

Machine Learning with Latent Variables

Idea

- There are **hidden variables** $\boldsymbol{z}$ behind the **observed data**

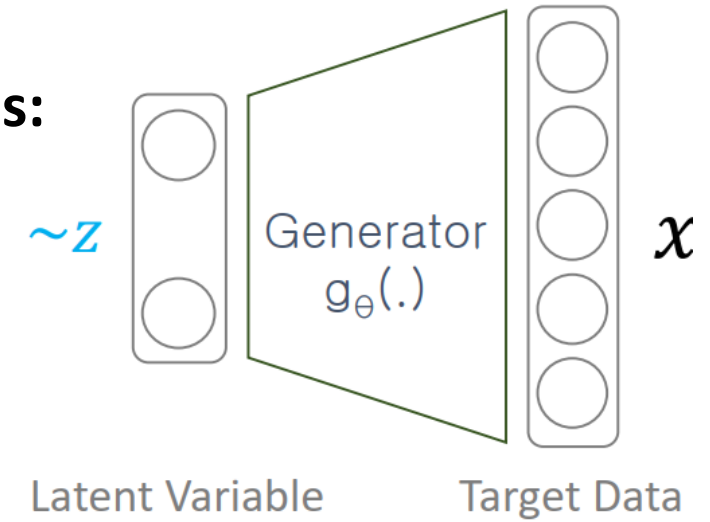


Machine Learning with Latent Variables

- **Latent variables z can be seen as a set of control parameters for generated data**
- **Under this assumption, the data distribution is:**

$$p(x) = \int p(x|z)p(z)dz$$

  - We cannot compute it directly. It is intractable.



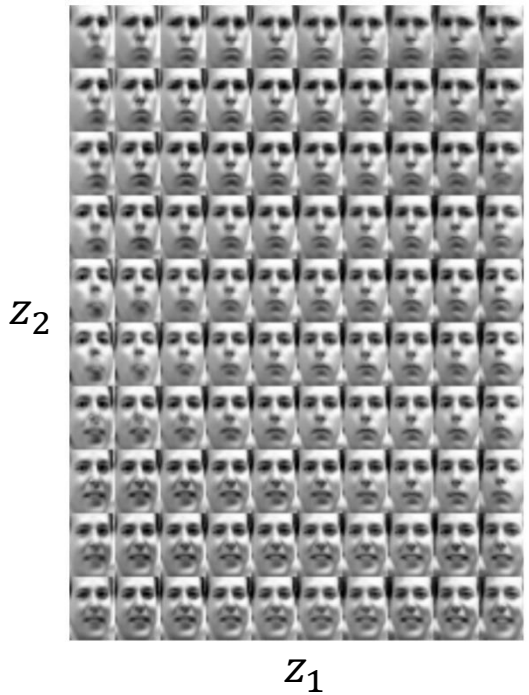
- **Variational Autoencoder defines three problems and solves them effectively via encoder-decoder structure:**
  - P1 how to define prior of ***latent variables*** z
    (i.e., decide what information they represent)
  - P2 how to define ***distribution*** $\boldsymbol{p(x|z)}$
  - P3 how to deal with ***integral over z***

P1: Definition of Prior $p(z)$

- **Choose prior $p(z)$ to be <u>simple</u>**
  - Reasonable for latent attributes (*e.g.*, pose, how much smile) (1)

- **Common choice: unit Gaussian $p(z) = \mathcal{N}(z; \mathbf{0}, \mathrm{I})$**
  - ***Diagonal*** prior ⇒ ***independent*** latent variables
  - Different dimensions of z encode interpretable factors of variation

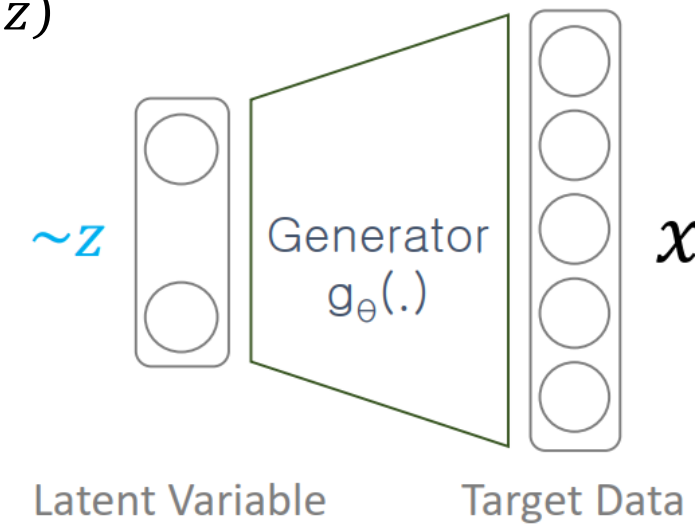
- **Example**
  - Varying z_1
    - Varying ***head*** pose
  - Varying z_2
    - Varying degree of ***smile***



P2: Definition of Distribution $p(x|z)$

- **Use common parametric distributions for $p(x|z)$**
  - Continuous x: a Gaussian parameterized by $g(z;\theta)$
  - Binary x: a Bernoulli parameterized by $g(z;\theta)$
- **Use a trainable neural network $g(z;\theta)$**
  - θ denotes network weights and biases of neural net
  - $g(z;\theta)$ produces parameters of $p(x|z)$

$z \sim p(z)$	Random variable
$g_\theta(\cdot)$	Deterministic function parameterized by θ
$x = g_\theta(z)$	Random variable



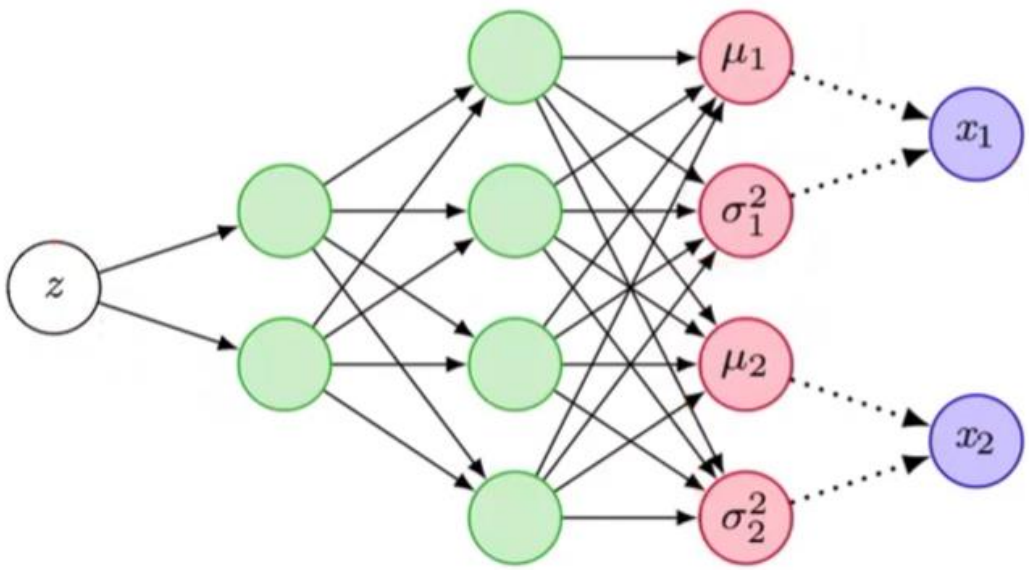
- **After training**
  - A new x is sampled from $p_\theta(x|z)$

P2: Definition of output distribution $p(x|z)$

2D example:

- We define $\boldsymbol{p}(\boldsymbol{x}|\boldsymbol{z})$ as Gaussian $\Rightarrow \mathcal{N}\big(\boldsymbol{x}; \boldsymbol{\mu}(\boldsymbol{z}), \boldsymbol{\Sigma}(\mathbf{z})\big)$
- $x \in \mathbb{R}^2,\ z \in \mathbb{R}$

$$\mu(z) = \begin{bmatrix} \mu_1(z) \\ \mu_2(z) \end{bmatrix}, \qquad \Sigma(z) = \begin{bmatrix} \sigma_1^2(z) & 0 \\ 0 & \sigma_2^2(z) \end{bmatrix}$$



$p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$: decoder

P3: Integration over z

- **For solution 1 & 2:**

$$p(x) = \int \underbrace{p(z)}_{\substack{\text{Simple} \\ \text{Gaussian} \\ \text{prior}}} \underbrace{p(x|z)}_{\substack{\text{Learn with} \\ \text{Neural net} \\ \text{“decoder”}}} dz$$

  - $p(z)$ and $p(x|z)$ are okay but ***integration remains <u>intractable</u>***
  - The number of z values can be extremely large in high-dimensional space

- **In practice: $\boldsymbol{p(x|z) \approx 0}$ for most $\boldsymbol{z \rightarrow}$ contribute nothing to estimate of $\boldsymbol{p(x)}$**

- **Key idea #1 for P3:**
  - Try to **sample** values of z that are **likely to have produced $\boldsymbol{x}$**, and compute $p(x)$ just from those!

    => We need Posterior distribution $\boldsymbol{p(z|x)}$

P3: Integration over z

- **However, estimating the exact posterior $p(z|x)$ is impossible**
  - The computation is <u>intractable:</u> $p(z|x) = p(x|z)p(z)/p(x)$

- **Key idea #2 for P3: employ variational inference!**
  - Find an approximate posterior $q(z|x) \approx p(z|x)$

- **This means: we need a new function $q(z|x)$, which can**
  - Take a value of x, and
  - Give us <u>a distribution over z that are likely to produce x</u>

P3: Integration over z

- **Key idea #3 for P3: use a <u>neural network $f(x;\phi)$</u>**
  - ϕ denotes parameters of the neural net (encoder)
  - $f(x;\phi)$ produces parameters of $q(z|x)$

$$q(z|x) = q\big(z; f(x;\phi)\big) \triangleq q_\phi(z|x)$$

  - Training the encoder can be done by the efficient techniques (e.g., SGD, backprop)

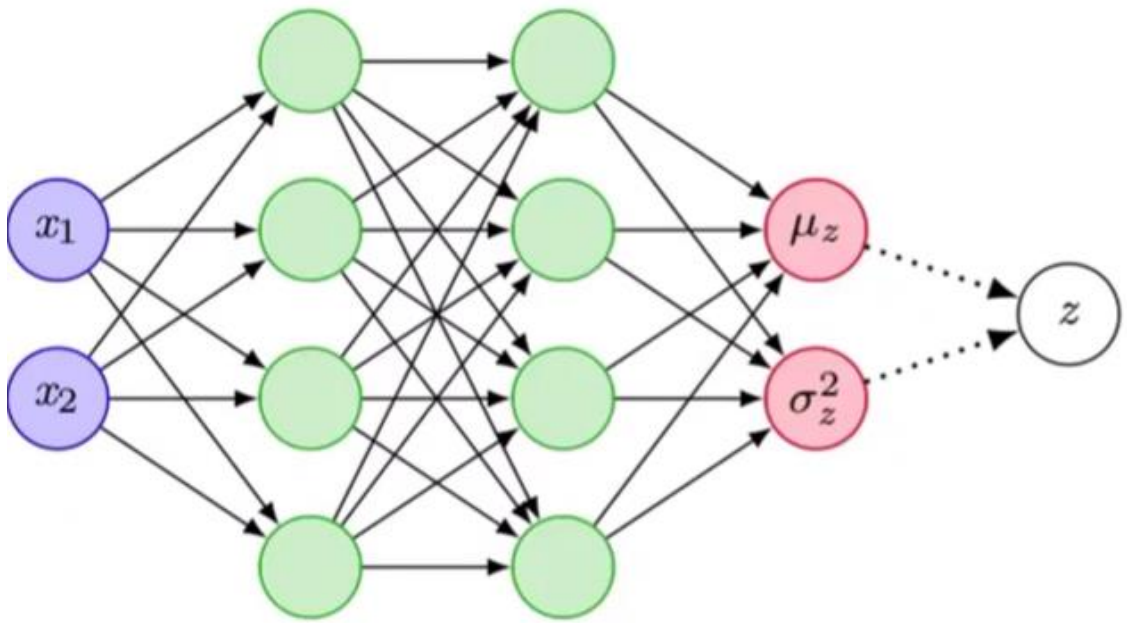
- **Common parametric distributions for $q(z|x)$**
  - A Gaussian parameterized by $f(x;\phi)$
  - Its covariance Σ: constrained to be ***diagonal*** (for computational issues)

- **Now, z is sampled from $q_\phi(z|x)$**

P3: Integration over z

2D example:

- We define $\boldsymbol{q}(\boldsymbol{z}|\boldsymbol{x})$ as Gaussian $\Rightarrow \mathcal{N}\big(\boldsymbol{x}; \boldsymbol{\mu}(\boldsymbol{x}), \boldsymbol{\Sigma}(\boldsymbol{x})\big)$
- $x \in \mathbb{R}^2,\ z \in \mathbb{R}$



$q_\phi(z \mid x)$: encoder

Variational Autoencoder

Setting an MLE problem:

- **log-likelihood** of training data x: $\log p(x)$

$$\log p(x) = \mathrm{E}_{z\sim q_\phi(z|x)}[\log p(x)] \quad \text{(Taking expectation with regards to } \mathbf{z}\text{)}$$

$$= \mathrm{E}_z[\log \frac{p(x|z)p(z)}{p(z|x)}] \quad \text{(Bayes' rule: } P(A|B) = \frac{P(B|A)P(A)}{P(B)}\text{)}$$

$$= \mathrm{E}_z[\log \frac{p(x|z)p(z)}{p(z|x)} \frac{q_\phi(z|x)}{q_\phi(z|x)}] \quad \text{(Multiply \& divide with the same term)}$$

$$= \boxed{\mathrm{E}_z[\log P_\theta(x|z)] - \mathrm{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right]} + \mathrm{E}_z[\log \frac{q_\phi(z|x)}{p(z|x)}]$$

$$= \underline{ELBO(\theta,\phi)} + \underline{D_{KL}(q_\phi(z|x)||\, p(z|x))}$$

Tractable lower bound

Intractable, so cannot compute this term, **but we know that KL-divergence always >= 0**

- Instead of minimizing D_{KL}, we will maximize the ELBO term!
  - This will approximate our $q_\phi(z|x)$ to the true posterior

Variational Autoencoder

☐ ELBO: $\mathcal{L}(\theta, \phi)$

$$\mathcal{L}(\theta, \phi) = \mathrm{E}_z[\log p_{\boldsymbol{\theta}}(x|z)] - \mathrm{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right]$$

$$= \mathrm{E}_z[\log p_{\boldsymbol{\theta}}(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$$

- $\log p(x) \geq \mathrm{E}_z[\log P_{\boldsymbol{\theta}}(x|z)] - D_{KL}(q_\phi(z|x)||p(z))$

☐ Our optimization problem:

$$\theta^*, \phi^* = \arg\max_{\theta,\phi} \sum_{i=1}^{N} \mathcal{L}(\theta, \phi)$$

- θ: parameters of generator network ("decoder")
- ϕ:parameters of inference network("encoder")
- N: number of training samples ($\{x^{(1)}, \dots, x^{(N)}\}$)

Thank You !